Informe Técnico: Módulo de Catálogo de Pergamino con Acceso JDBC

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Esta evidencia codifica un módulo del proyecto **MONK GRAIN** con conexión a base de datos por **JDBC**, implementando las cuatro operaciones que pide la guía: inserción, consulta, actualización y eliminación.

El módulo elegido es el **catálogo de lotes de café pergamino**, que es el corazón del producto: lo que un caficultor publica y lo que un comprador examina antes de decidir. Se construye sobre los artefactos previos del proyecto —el diagrama de clases, las historias de usuario y el modelo de datos— y respeta los estándares de codificación definidos en la evidencia AA1-EV02.

2. Objetivo

Codificar un módulo del software con conexión a base de datos mediante JDBC, que implemente inserción, consulta, actualización y eliminación, cumpliendo los estándares de nombres de variables, métodos, clases y paquetes, y versionándolo con Git.

3. El dominio: por qué el catálogo de pergamino

El pergamino seco es el café despulpado, fermentado, lavado y secado que todavía conserva la película que envuelve la almendra y **no ha pasado por trilla**. Es el producto con el que un caficultor colombiano sale de su finca.

Esa distinción no es terminológica, es económica: un lote de pergamino **no se describe por cómo sabe en taza**, sino por cuánto café exportable rinde al trillarlo. De ahí que el atributo que gobierna su valor sea el **factor de rendimiento** y no un puntaje de catación.

Factor	Lectura	Significado comercial
≤ 89	Excepcional	Rinde mucho más que el estándar; se paga sobreprecio
90 – 92	De exportación	Por debajo del estándar; apto para trilla de excelso
93 – 96	Estándar	El rango de la compra corriente
≥ 97	Bajo rendimiento	Hay que trillar más pergamino para el mismo excelso

> **Menos es mejor.** Es la inversión que más se presta a equivocarse al programar: un factor más bajo debe producir una etiqueta mejor, y el filtro del comprador acota el **techo** que acepta, no el mínimo. Hay una prueba dedicada a fijar esa inversión.

4. Arquitectura del módulo

Se aplica una separación en capas con una dirección única de dependencias, la misma que rige el resto del proyecto:

```
MainCatalog  →  CatalogService  →  DAO  →  JDBC  →  PostgreSQL
 (interfaz)     (reglas de          (SQL)
                 negocio)
```

Paquete	Responsabilidad	Qué NO hace
`model`	Objetos de dominio y lecturas derivadas (factor, humedad, cargas)	No conoce JDBC ni pantallas
`dao`	La **única** capa que escribe SQL	No contiene reglas de negocio
`service`	Reglas del negocio y validación	No sabe que detrás hay SQL
`util`	Conexión y excepciones propias	No depende del dominio

Esa división permite probar las reglas de negocio **sin levantar PostgreSQL** —las 13 pruebas del módulo no tocan la base— y cambiar el motor o pasar a un ORM sin tocar una sola regla.

5. Modelo de datos

Dos tablas, en una base propia (`monkgrain_java`) y no la de la aplicación: un módulo de aprendizaje que inserta y borra no debe tocar los datos de demostración del producto.

```
CREATE TABLE pergamino_lots (
    lot_id            SERIAL        PRIMARY KEY,
    producer_id       INTEGER       NOT NULL,
    lot_code          VARCHAR(60)   NOT NULL,
    variety           VARCHAR(80),

    -- Análisis físico. Todo opcional: exigirlo para publicar
    -- dejaría fuera a la mayoría de caficultores pequeños.
    yield_factor      NUMERIC(5,2),
    moisture_pct      NUMERIC(4,2),
    healthy_bean_pct  NUMERIC(5,2),
    pasilla_pct       NUMERIC(5,2),
    borer_damage_pct  NUMERIC(5,2),

    price_per_kg      NUMERIC(14,2) NOT NULL,
    available_kg      INTEGER       NOT NULL,
    harvest_date      DATE,
    published         BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at        TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_lot_producer FOREIGN KEY (producer_id)
        REFERENCES producers (producer_id) ON DELETE RESTRICT,

    CONSTRAINT uq_lot_code_per_producer UNIQUE (producer_id, lot_code),

    -- Fuera del rango 80-130 no existe café comercializable.
    CONSTRAINT ck_yield_factor_range
        CHECK (yield_factor IS NULL OR yield_factor BETWEEN 80 AND 130),

    CONSTRAINT ck_price_non_negative     CHECK (price_per_kg >= 0),
    CONSTRAINT ck_available_non_negative CHECK (available_kg >= 0)
);
```

5.1 Decisiones del modelo y su motivo

Decisión	Por qué
`NUMERIC` y nunca `FLOAT` para precios	En coma flotante, 0,1 + 0,2 no es 0,3. Sobre una carga de 125 kg ese error se vuelve dinero real
Todo el análisis físico es opcional	Exigirlo para poder publicar dejaría fuera a la mayoría de caficultores pequeños, que no han mandado su café a un laboratorio
`CHECK` en la base, no solo en Java	La regla se cumple venga la escritura de donde venga, incluso desde una consola SQL
`UNIQUE (producer_id, lot_code)`	El código identifica el lote dentro de la finca; repetirlo haría ambigua cualquier reclamación
`ON DELETE RESTRICT`	Borrar un vendedor con lotes publicados dejaría lotes huérfanos

6. Las cuatro operaciones

6.1 Inserción

Con `PreparedStatement` y parámetros, nunca concatenación de cadenas. `RETURN_GENERATED_KEYS` evita la consulta adicional que haría falta para averiguar el identificador recién creado:

```
public int insert(PergaminoLot lot) {
    try (Connection connection = DatabaseConnection.open();
         PreparedStatement statement =
                 connection.prepareStatement(INSERT_SQL, Statement.RETURN_GENERATED_KEYS)) {

        statement.setInt(1, lot.getProducerId());
        statement.setString(2, lot.getLotCode());
        setDecimal(statement, 4, lot.getYieldFactor());
        statement.setBigDecimal(9, lot.getPricePerKg());
        statement.setInt(10, lot.getAvailableKg());
        statement.executeUpdate();

        try (ResultSet keys = statement.getGeneratedKeys()) {
            if (keys.next()) {
                int id = keys.getInt(1);
                lot.setLotId(id);
                return id;
            }
        }
        throw new DataAccessException("La base de datos no devolvió el identificador genera-
do.");

    } catch (SQLException e) {
        throw new DataAccessException(
                "No fue posible registrar el lote " + lot.getLotCode() + ".", e);
    }
}
```

6.2 Consulta con filtro

El filtro del catálogo acota el factor por su **máximo**. El mismo parámetro viaja dos veces —una para el `IS NULL` que desactiva el filtro y otra para la comparación—, de modo que una sola consulta sirve con filtro y sin él:

```
SELECT ... FROM pergamino_lots
WHERE published = TRUE AND available_kg > 0
  AND (? IS NULL OR yield_factor <= ?)
ORDER BY yield_factor ASC NULLS LAST, price_per_kg ASC
```

El orden es ascendente porque menos es mejor, y los lotes sin factor declarado quedan al final: un lote que no declara nada no puede afirmar que cumple nada.

6.3 Actualización

No se permite cambiar el productor ni el código del lote: son su identidad comercial, y moverlos dejaría sin sentido las reclamaciones ya presentadas contra él. El método devuelve si la fila existía:

```
public boolean update(PergaminoLot lot) {
    try (Connection connection = DatabaseConnection.open();
         PreparedStatement statement = connection.prepareStatement(UPDATE_SQL)) {
        ...
        return statement.executeUpdate() > 0;
    } catch (SQLException e) {
        throw new DataAccessException(
                "No fue posible actualizar el lote " + lot.getLotId() + ".", e);
    }
}
```

6.4 Eliminación

```
public boolean delete(int lotId) {
    try (Connection connection = DatabaseConnection.open();
         PreparedStatement statement = connection.prepareStatement(DELETE_SQL)) {

        statement.setInt(1, lotId);
        return statement.executeUpdate() > 0;

    } catch (SQLException e) {
        throw new DataAccessException("No fue posible eliminar el lote " + lotId + ".", e);
    }
}
```

7. Prácticas de JDBC aplicadas

Práctica	Por qué
`PreparedStatement` con parámetros, **siempre**	Concatenar la entrada del usuario dentro del SQL es exactamente lo que permite una inyección, y basta con que se olvide una vez
`try-with-resources` en cada operación	Cierra conexión y sentencia aunque haya excepción. Una conexión que se queda abierta agota el cupo del servidor y el fallo aparece después, en una operación que no tiene la culpa
`setNull` con el tipo explícito	`setBigDecimal(null)` funciona en PostgreSQL pero no en todos los controladores
Excepciones propias que envuelven `SQLException`	Las capas superiores no tienen que declararla en cada firma; y se conserva siempre la causa original, sin la cual depurar es adivinar
`Optional` en vez de `null`	Obliga a decidir qué hacer cuando no existe, en vez de dejar que aparezca un `NullPointerException` tres capas más arriba
Credenciales fuera del código	Una credencial dentro de una clase viaja al repositorio y no se cambia sin recompilar

8. Estándares de codificación aplicados

Los definidos en AA1-EV02, en su equivalencia para Java:

Elemento	Convención	Ejemplo
Paquetes	minúsculas, dominio invertido	`co.monkgrain.catalog.dao`
Clases	`PascalCase`	`PergaminoLotDao`
Métodos y variables	`camelCase`, verbo primero	`findPublished()`
Constantes	`SCREAMING_SNAKE_CASE`	`KILOS_PER_CARGA`
Tablas y columnas	`snake_case`, tabla en plural	`pergamino_lots`
Idioma	código en inglés, comentarios en español	`getYieldFactorLabel()`

Los comentarios explican **el porqué, no el qué**. El código ya dice qué hace; lo que no puede decir es qué alternativa se descartó y por qué razón.

9. Ejecución verificada

Compilación y pruebas:

```
$ mvn clean package
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: target/catalogo-pergamino.jar
[INFO] Building jar: target/catalogo-pergamino-jar-with-dependencies.jar
[INFO] BUILD SUCCESS
```

Ejecución del CRUD completo contra PostgreSQL:

```
$ java -jar target/catalogo-pergamino-jar-with-dependencies.jar

 MONK GRAIN — Catálogo de pergamino (JDBC)
Base de datos: jdbc:postgresql://localhost:5432/monkgrain_java

 1. INSERCIÓN — registrar el vendedor
Vendedor registrado: [1] Finca El Mirador — Pitalito, Huila

 2. INSERCIÓN — publicar lotes de pergamino
Lote registrado: [1] MIR-2026-001 — factor 88.50 (Excepcional) — 1250 kg (10 cargas) a
24500.00/kg
  Lectura del factor : Excepcional
  Humedad en rango   : sí
  Valor del lote     : 30625000.00
Lote registrado: [2] MIR-2026-002 — factor 94.00 (Estándar) — 375 kg (3 cargas) a 19800.00/
kg
Lote registrado: [3] MIR-2026-003 — factor — (Sin declarar) — 80 kg a 17200.00/kg
  Sin análisis de laboratorio, y aun así publicable.

 3. CONSULTA — todos los lotes del vendedor
Lotes registrados: 3

 4. CONSULTA — catálogo filtrado por factor máximo 92
Lotes que cumplen: 1
  [1] MIR-2026-001 — factor 88.50 (Excepcional) — 1250 kg (10 cargas) a 24500.00/kg

 5. ACTUALIZACIÓN — corregir el precio y descontar una venta
Antes : [1] MIR-2026-001 — ... a 24500.00/kg
Después: [1] MIR-2026-001 — ... a 25800.00/kg

Venta de 2 cargas (250 kg):
  [1] MIR-2026-001 — factor 88.50 (Excepcional) — 1000 kg (8 cargas) a 25800.00/kg
```

9.1 Validaciones ejercidas

```
 6. VALIDACIONES — lo que el módulo rechaza
  [ok] Factor de rendimiento imposible (145)
       → El factor de rendimiento debe estar entre 80 y 130. Un valor fuera de
         ese rango es un error de digitación, no un café excepcional.
  [ok] Código de lote repetido
       → Ya tienes un lote con el código MIR-2026-001.
  [ok] Precio negativo
       → El precio por kilogramo no puede ser negativo.
  [ok] Humedad del 150 %
       → La humedad debe estar entre 0 % y 100 %.
  [ok] Vender más kilos de los que hay
       → Solo quedan 1000 kg disponibles.
```

9.2 Eliminación y estado final

```
 7. ELIMINACIÓN — retirar los lotes de la demostración
Eliminado: MIR-2026-001
Eliminado: MIR-2026-002
Eliminado: MIR-2026-003
Eliminado el vendedor 1

Lotes restantes      : 0
Vendedores restantes : 0
El lote 1 ya no se encuentra: true
```

> **La demostración es idempotente.** Deja la base como la encontró, de modo que puede ejecutarse las veces que haga falta sin acumular datos. Comprobado ejecutándola dos veces seguidas: la base queda en 0 vendedores y 0 lotes.

10. Pruebas

Trece pruebas con JUnit 5 sobre las lecturas del dominio. No tocan la base de datos a propósito: son reglas del negocio y deben poder comprobarse sin levantar PostgreSQL.

Grupo	Qué fija
Factor de rendimiento (6)	Los cuatro tramos, que un lote sin factor no afirme nada, y **que menos sea mejor**
Humedad (4)	El rango seguro de 10 % a 12 %, y que sin el dato no se pueda afirmar que está en rango
Cantidades y valor (3)	La lectura en cargas, que por debajo de una no se convierta, y que el valor no use coma flotante

11. Conclusión

El módulo `catalogo-pergamino` implementa las cuatro operaciones del CRUD sobre el catálogo de lotes mediante JDBC, con separación en capas, validación en dos niveles —aplicación y base de datos— y las prácticas que evitan los dos errores más caros de JDBC: la inyección de SQL y las conexiones que no se cierran.

Se verificó en ejecución contra PostgreSQL: 13 pruebas en verde, el ciclo completo recorrido y la base devuelta a su estado inicial.